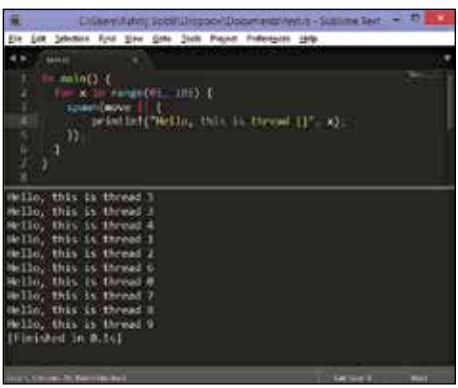
While Go is new, and publicly developed with community contributions, Rust is in an earlier and more exciting phase where there is more scope for change, and backward compatibility isn't much of an issue. For someone who wants to be involved in the development of a programming language, this is a perfect opportunity. Even if you not interested in this aspect of programming, seeing the development of a language can give you insight into the programming language you currently use.

Even before releasing a first stable version, or reaching a stable specification, it has begun to influence other languages. For instance it was one of the language that influenced Swift, Apple's recently released programming language for iOS and OSX development.

The most interesting thing about Rust is that it is incredibly simple to do seemingly complex things. For example creating concurrent applications that run multiple threads where each works on bits of the same problem while coordinating with a main thread; this can be a pain in many languages. However solving this challenge is something that Rust is built around and as such you might find the syntax for doing such things simpler than in many other languages.

For example if you wanted to simply create an application that prints "Hello World" from a separate thread, it would be as simple as: 'spawn(move || {println!("Hello, world! {}", x);});' Note that the above syntax may change (it already has from the stable 0.12 to 0.13 in development).



The above simple code sample runs each iteration of the loop in a separate thread. Each time you run this code, the order of the output will be different depending on the order in which the threads finish executing.

Rust also has an intricate system for controlling access to pointers such that you never end up in a situation that causes crashes due to incorrect use of pointer. It's also better at tracking pointer memory usage without major performance penalties.

Finally, Rust also makes it easy to integrate with legacy C code with efficient C bindings.